

OS1B - Testing Plan

Augmented Security Knowledge Base

Team Members:

- Cong Deng a1155047
- Sheng Wang a1903948
- Tianhua Zhang a1915934
- Xin Wei a1912958
- Yifan Gu a1909803

Supervisor: Orvila Sarker

Table of Contents

1. Introduction.....	1
2. SVM and LDA Model Testing	1
3. Similarity Accuracy Testing	1
3.1 Testing Methods.....	1
3.2 Outcomes.....	1
3.3 Test Cases	2
4. User Interface and Interaction Testing	2
4.1. Testing Methods.....	2
4.2. Expected Outcomes.....	2
4.3 Test Cases	2
5. Functional Logic Testing (Frontend & Backend)	3
5.1. Testing Methods.....	3
5.2. Expected Outcomes.....	3
5.3. Test Cases	3
6. Defect Management	3
7. Testing Schedule.....	4
Appendix	5
Appendix 1:	5
Appendix 2:	6
Appendix 3	9
Appendix 4	10

1. Introduction

This project aims to build a system for automatically handling computer security issues. The system supports question answering by combining the LLM model and search through existing related data in the dataset to build a question answering system. This system can automatically generate an answer to user input and then return relevant data in the dataset as a reference.

2. SVM and LDA Model Testing

Testing Methods	Testing Result
Use logistic regression method instead of SVM for testing	Appendix 1
Test with 100 labelled external data generated by ChatGPT 4o	Appendix 1

3. Similarity Accuracy Testing

3.1 Testing Methods

We designed a two-stage evaluation process. First, we generate paraphrased versions of original security-related Stack Overflow questions using the Llama3 language model. These paraphrases simulate varied user expressions while preserving the original intent, creating an input set for testing.

In the second stage, we have set multiple semantic search configurations. Retrieval strategies (e.g., cosine similarity, Maximal Marginal Relevance, and threshold filtering) are tested alongside different sentence embedding models (e.g., all-MiniLM-L6-v2, paraphrase-multilingual-MiniLM-L12-v2, all-mpnet-base-v2). Each configuration is assessed using a fixed corpus and the generated paraphrased queries.

Performance checks include Top-1/2/3 match accuracy (whether the original question is retrieved), and average query speed. All tests are run in a same environment.

3.2 Outcomes

We expect this setting shows how different configurations can handle paraphrased real-world-like user questions. Ideal systems will maintain high accuracy across variations while remaining time-efficient.

The results will help identify which retrieval strategies and embeddings offer the best trade-off between performance and speed. This enables informed decisions when optimizing semantic search systems for real-world QA applications.

3.3 Test Cases

[Appendix 2](#)

4. User Interface and Interaction Testing

4.1. Testing Methods

Module	Methods	Tools
User Interface Testing	Component interaction testing, User journey testing	Cypress, React Testing Library, TestCafe
Responsive Interaction Testing	Device emulation, Breakpoint testing, Orientation change testing	Chrome DevTools, Responsive Design Mode, Throttling tools
Performance-Related Interaction Testing	Response time measurement, Input latency testing, Resource monitoring	React Profiler, Chrome/Edge DevTools, WebPageTest

4.2. Expected Outcomes

Module	Criteria	Success Metrics
Navigation	Fast response, correct history & breadcrumbs	State updates $\leq 100\text{ms}$, correct breadcrumbs & back/forward behaviour
Form & Input	Clear feedback and validation	Instant feedback, real-time errors, visible submit status
Loading & Progress	Show loading and progress clearly	Feedback $\leq 300\text{ms}$, indicators accurate & timely

4.3 Test Cases

[Appendix 3](#)

5. Functional Logic Testing (Frontend & Backend)

5.1. Testing Methods

Frontend	
Method 1	Use React Testing Library with Jest to simulate state transitions based on mocked API responses.
Method 2	Use msw (Mock Service Worker) to intercept network calls and simulate backend behaviour during tests.
Method 3	Verify React hooks and Zustand state updates based on backend responses
Backend	
Method 1	Use pytest with httpx.AsyncClient or TestClient from FastAPI for integration testing.
Method 2	Mock external dependencies using unittest.mock or pytest-mock.
Method 3	Validate response structure, status codes, and LLM result content.

5.2. Expected Outcomes

Backend endpoints return correct responses for valid inputs and handle edge cases (400/500) gracefully.

Frontend correctly triggers backend requests and reflects results in UI and component state.

Errors or delays are properly shown in frontend with fallback UIs or retry options.

5.3. Test Cases

[Appendix 4](#)

6. Defect Management

- Use an issue tracker (GitHub Issues in our case) to record all defects.
- Each defect record should include: module, description, reproduction steps, severity level, and status (New / In Progress / Fixed).

7. Testing Schedule

Phase	Timeframe	Testing Content
Weeks 8-9	Unit Testing & Performance/Stress Testing	Validation of data processing flow, classifier functionality and performance, UI & functional testing, and system performance under large-scale data input
Week 10	Regression Testing	Verify that all existing functions and AI model outputs remain stable and correct after updates, including: • SVM classification accuracy and LDA topic interpretability • ChatGPT & LLM summary and skill extraction quality • End-to-end pipeline integration stability • Frontend data display correctness
Week 11	UAT (User Acceptance Testing)	Final evaluation of product by team members

Appendix

Appendix 1:

- Evaluation Results: SVM and Logistic Regression

Performance Evaluation – Logistic Regression					Performance Evaluation – SVM				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Conceptual	0.54	0.71	0.61	21	Conceptual	0.59	0.62	0.60	21
Debugging	0.77	0.72	0.74	60	Debugging	0.83	0.87	0.85	60
How-to	0.48	0.44	0.46	32	How-to	0.64	0.56	0.60	32
accuracy			0.64	113	accuracy			0.73	113
macro avg	0.60	0.62	0.60	113	macro avg	0.69	0.68	0.68	113
weighted avg	0.64	0.64	0.64	113	weighted avg	0.73	0.73	0.73	113

Precision: Among the samples predicted as a certain class by the classifier, the proportion that actually belongs to that class.

Recall: Among the samples that truly belong to a certain class, the proportion correctly identified by the classifier.

F1-score: The harmonic mean of precision and recall. It serves as a balanced measure of classification performance.

Support: The actual number of instances of each class in the test dataset.

Accuracy: Overall proportion of correctly classified samples. Reflects the model's general correctness.

Macro Avg: Unweighted average of precision, recall, and F1-score across all classes. Each class is equally important. Good for assessing fairness.

Weighted Avg: Average of precision, recall, and F1-score, weighted by class support. Larger classes contribute more. Suitable for imbalanced datasets.

- Model Performance on Internal vs. External Test Sets

===== Original dataset test performance =====					===== External test set performance =====				
	precision	recall	f1-score	support		precision	recall	f1-score	support
Conceptual	0.59	0.62	0.60	21	Conceptual	0.69	0.97	0.80	38
Debugging	0.83	0.87	0.85	60	Debugging	1.00	1.00	1.00	30
How-to	0.64	0.56	0.60	32	How-to	0.94	0.47	0.62	32
accuracy			0.73	113	accuracy			0.82	100
macro avg	0.69	0.68	0.68	113	macro avg	0.87	0.81	0.81	100
weighted avg	0.73	0.73	0.73	113	weighted avg	0.86	0.82	0.81	100

Appendix 2:

there are two key modules:

1. question_generate.py

Generates paraphrased versions of original questions using the Llama3 language model.

Produces a final file (rewritten_questions_final.csv) used to simulate user input in real life situations.

Some question example are:

original_question	alternative_question
I'm trying to figure out if the security quality of SSL can be mimicked without the need for a centralized CA. There are many questions on here about setting up pure-JS diffie hellman in place of SSL. All of them are roundly refuted, and I think I understand why (although I completely disagree that there's no value in DH-keyed symmetric encryption for the millions of sites which are not going to be implementing full SSL anytime soon either way). Anyway, question is this: it seems as though the only preventative reason JS DH wouldn't work is that you lose authentication so you're still subject to man-in-the-middle. All the myriad other issues flow from attempts to resolve this one. But what if you used a blockchain as a certificate authority? The process would then be Form DH key encrypt any handshake string with known public key from blockchain encrypt this verify string with DH key if handshake string received back, authentication verifies and DH is as secure as SSL DH Edit: didn't mean to imply DH was symmetric, adjusted for clarity	Can a decentralized alternative to a traditional Certificate Authority (CA) be designed using blockchain technology, effectively mimicking the security quality of SSL without relying on a centralized entity? This rewritten question maintains the same meaning and intent as the original while using different wording and structure. It also makes it clear that you're exploring an alternative to traditional CA's, which is the core idea behind your proposal to use a blockchain as a certificate authority.
I'm using a PKCS11 smartcard at work, and would like to use jarsigner to sign jar files using the certificate on my card. I'm mostly working on Linux. Coolkey can see the card. Oracle's documentation mentions smartcards: <pre>jarsigner -keystore NONE -storetype PKCS11 \ -providerName SunPKCS11-SmartCard \ -list</pre> But there's apparently no actual provider by that name, and the -list argument to jarsigner doesn't appear to exist. I finally got jarsigner to see the card, but it reports that it isn't a valid entry and doesn't have a private key: <pre>jarsigner -keystore NONE -storetype PKCS11 \ -providerClass sun.security.pkcs11.SunPKCS11 \ -providerArg smartcard.config \ -storepass notayrealpass \ myjarfile.jar 'Identity #0'</pre> jarsigner: Certificate chain not found for: Identity #0. Identity #0 must reference a valid KeyStore key entry containing a private key and corresponding public key certificate chain. Where smartcard.config is: <pre>name=Kittens library=/usr/lib/pkcs11/libcoolkeypk11.so</pre>	"I'm trying to use a smartcard-based certificate stored on my PKCS11 card to sign JAR files with jarsigner. I've successfully configured coolkey to recognize the card, and Oracle's documentation suggests that jarsigner supports PKCS11 storetypes. However, when I try to list available certificates or sign a JAR file using the 'Identity #0' alias, I get an error indicating that the certificate chain is not found. I've tried various combinations of options, including specifying the provider class and argument, but nothing seems to work. The issue doesn't seem to be with loading the card's configuration or accessing its aliases programmatically in Java - I can retrieve a list of available identities just fine. Given that the private key should remain on the smartcard and signing should occur on the card itself, is it possible to use jarsigner for this purpose? Alternatively, are there other tools or approaches that might allow me to sign JAR files with a PKCS11-based certificate?"

For more question, please check in github :

[OS1B/Data/rewritten_questions_final.csv at main · MCI-Project-2025/OS1B](#)

2. benchmark_retrieval_configurations.py

Benchmarks different embedding models and retrieval strategies.

Evaluates three search types: similarity, mmr, and similarity_score_threshold.

Tests three sentence embedding models (all-MiniLM-L6-v2, paraphrase-multilingual-MiniLM-L12-v2, all-mpnet-base-v2).

Measures Top-1/2/3 accuracy and average query time.

To assess the precision of our system, we track the exact rank position (1, 2, or 3) of the correct original question among the top retrieved candidates for each paraphrased input.

The results are reported as the proportion of cases where the correct match appears in **First_position**, **second_position**, **Third_position**, or **not found** in the top 3.

- **First_position** measures the proportion of paraphrased questions for which the original version match exact in rank position 1 (the first question in returned question set)
- **second_position** measures the proportion of paraphrased questions for which the original version match exact in rank position 2 (the second question in returned question set)
- **Third_position** measures the proportion of paraphrased questions for which the original version match exact in rank position 3 (the third question in returned question set)
- **not found** measures the proportion of paraphrased questions for which the original version are not in returned question set

Overall result is:

- Search Type: Similarity(cosine_similarity)

	sentence-transformers/ all-MiniLM-L6-v2	sentence-transformer s/all-mpnet-base-v2	sentence-transformer s/paraphrase-multilin gual-MiniLM-L12-v2
first_position	97.86%	97.86%	97.86%
second_position	1.06%	1.06%	1.06%
third_position	0.0%	0.0%	0%
not_found	1.06%	1.06%	1.06%

- Search Type: Similarity_score_threshold

	sentence-transformers/ all-MiniLM-L6-v2	sentence-transformer s/all-mpnet-base-v2	sentence-transformer s/paraphrase-multilin gual-MiniLM-L12-v2
first_position	92.78%	92.78%	93.04%
second_position	0.0%	0.0%	0.0%
third_position	0.0%	0.26%	0.0%
not_found	7.21%	6.95%	6.95%

- Search Type: mmr(Maximal Marginal Relevance)

	sentence-transformers/ all-MiniLM-L6-v2	sentence-transformer s/all-mpnet-base-v2	sentence-transformer s/paraphrase-multilin gual-MiniLM-L12-v2
first_position	97.86%	97.86%	97.86%
second_position	0.53%	0.53%	0.53%
third_position	0.0%	0.0%	0.0%
not_found	1.6%	1.60%	1.60%

Outputs results in model_comparison_results.json.

[OS1B/Data/model_comparison_results.json at main · MCI-Project-2025/OS1B](#)

These modules support evaluating both semantic matching fidelity and system performance under paraphrased input variations.

Appendix 3

User Interface Test Cases

Test Case ID	Test Target	Description	Tools	Expected Outcome
UI-001	Search Button Interaction	Test visual feedback on search button hover/click and loading state appearance	Jest, React Testing Library	Button changes on hover; shows pressed state on click; displays loading animation during search
UI-002	Search Input Keyboard Interaction	Test keyboard interactions and focus indicators for search input	Jest, React Testing Library	Enter key starts search; Tab navigation shows clear focus indicator
UI-003	Search Loading State	Test skeleton screen animations during search operations	React Testing Library, Jest	Skeleton appears within 100ms; animations run smoothly; transitions to results smoothly
UI-004	Answer Component Interaction	Test Answer component's expand/collapse, link clicks and source reference viewing	Cypress, React Testing Library	Content expands/collapses smoothly; links have hover styles; source references show tooltips
UI-005	Post Content Navigation	Test navigation through related posts in PostContent and browser history preservation	Cypress	Post links show hover states; clicking navigates correctly; back button returns to previous post
UI-006	Team Modal Focus Management	Test keyboard focus management in TeamModal component	React Testing Library, Jest	Focus trapped in modal when open; Tab cycles through modal elements; focus returns to trigger on close
UI-007	Team Modal Animations	Test TeamModal opening/closing animations and backdrop interaction blocking	Jest, Chrome DevTools	Modal animates smoothly; backdrop fades in; clicking backdrop closes modal; background non-interactive
UI-008	Responsive Layout Transitions	Test UI component adaptations between different viewport sizes	Chrome DevTools, Responsive Design Mode	Layout transitions smoothly; no horizontal overflow; content readable at all sizes; no overlaps
UI-009	Keyboard Navigation Flow	Verify logical tab order and keyboard accessibility of interactive elements	axe-core, Manual Testing	Tab order follows visual layout; all interactive elements focusable; focus indicators clearly visible
UI-010	Screen Reader Announcements	Test screen reader notifications for dynamic content changes in Answer component	NVDA, VoiceOver	Loading state changes announced; search results appearance announced; expand/collapse states announced
UI-011	Search Results Rendering Performance	Measure performance when rendering large result sets in Answer component	Chrome DevTools Performance Panel, React Profiler	UI remains responsive during rendering; animations maintain ≥ 30 fps; scrolling smooth without jank
UI-012	Network Throttling Resilience	Test application behavior under slow network conditions	Chrome DevTools Network Throttling	Loading indicators appear promptly; partial content usable; error states shown after timeout
UI-013	Browser History Management	Verify browser navigation buttons maintain state after viewing results and details	Cypress	Back button returns to search results; forward returns to post details; no state resets

Appendix 4

- Frontend

Test Case ID	Test Target	Description (Goal)	Tools / Libraries	Expected Outcome
TC4.1	services/searchService.js	Verify the search API returns valid JSON response	jest, msw, axios-mock-adapter	Mocked API returns valid response; no exception
TC4.2	components/Answer.jsx	Ensure component renders API-provided search results	React Testing Library, jest, msw	DOM displays result elements after query
TC4.3	components/Filter.jsx	Filter interaction triggers API with correct query params	React Testing Library, msw	Results update based on filter selection
TC4.4	components/OriginalQuestion.jsx	Question input updates state and triggers API call	React Testing Library, jest	State is updated; API is triggered appropriately
TC4.5	components/Skeleton.jsx	Display loading state during API calls	React Testing Library, jest	Skeleton/loading UI is shown during async request
TC4.6	App.jsx	Show fallback/error UI on API failure	React Testing Library, msw	Error or retry UI shown instead of broken screen

- Backend

Test Case ID	Test Target	Description (Goal)	Tools / Libraries	Expected Outcome
TC4.7	/ root endpoint	Verify FastAPI service is running and responsive	pytest, TestClient	Returns status 200 with JSON: {"message": "Hello, FastAPI!"}
TC4.8	/askQuestion POST endpoint	Send a question and ensure it correctly invokes get_rag_response()	pytest, TestClient, mock	Returns 200 with a valid RAG-generated response JSON

TC4.9	get_rag_response() logic	Test the full RAG pipeline (retriever, generator, reranker) invocation logic	pytest, unittest.mock	returns a structured response with answer, sources
TC4.10	Input validation (POST)	Send empty or invalid question field and test Pydantic schema enforcement	pytest, TestClient	Returns with appropriate validation error message
TC4.11	Retriever module (retriever_builder.py)	Verify it retrieves top-k context passages properly	pytest, mock	Returns expected top-k results
TC4.12	Generator module (generator_builder.py)	Validate that it generates coherent answers from retrieved content	pytest, mock, tempfile	Returns a well-formed answer string of reasonable length